



**PREMIER UNIVERSITY, CHITTAGONG**  
**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (CSE)**

# **PROJECT REPORT**

**COURSE NAME:**

BIG DATA ANALYTICS LABORATORY (CSE 4346)

**PROJECT NAME:**

BIG DATA ANALYSIS OF UNIVERSITY STUDENT RECORDS

**SUBMITTED BY:**

**SHAISHOB BOIDYA**

ID: 0222220005101049

SECTION: B

SEMESTER: 8<sup>th</sup>

SESSION: SPRING 2026

BATCH: 42<sup>nd</sup>

**SUBMITTED TO:**

**MD. TAMIM HOSSAIN**

LECTURER, DEPARTMENT OF CSE  
PREMIER UNIVERSITY, CHITTAGONG

# 1. Introduction

This project applies Hadoop MapReduce and Apache Pig to a university student performance dataset in order to extract meaningful statistics about student enrolment, academic outcomes, and scholarship distribution across departments. The dataset contains detailed academic records of 150,000 students, including their department, semester, attendance, marks, grade, GPA, CGPA and scholarship status. Instead of relying on a single machine to process such a large volume of records, the project uses the Hadoop Distributed File System (HDFS) for storage and Hadoop's distributed processing engines for computation.

The main purpose of the project is to demonstrate that large tabular datasets can be processed efficiently and reliably using Big Data tools, and to compare two different approaches to distributed processing: low-level MapReduce jobs and high-level Pig Latin scripts. The project focuses on the following analytical questions:

- How many students are enrolled in each department?
- What is the average academic performance (Total Marks) of students in each department?
- How many students receive a scholarship, and how many do not?
- What is the average CGPA of students in each department?
- How are students distributed across different letter grades?
- Which departments have the highest average attendance?
- How does scholarship distribution differ between male and female students?
- How does average academic performance change across semesters?

To answer these questions, three MapReduce jobs and five Apache Pig scripts were designed and executed on a Hadoop cluster running inside Docker containers.

*The complete source code, mapper/reducer scripts, Pig scripts, analysis outputs and this report are publicly available in the project's GitHub repository: <https://github.com/SB-Abhi1/bda-university-student-analysis>*

## 2. Dataset Description

The dataset used in this project is a synthetic university student record dataset, **university\_student\_dataset.csv**, containing academic and demographic information for 150,000 students across fifteen departments. Each row of the dataset corresponds to one enrolled course record for a student and includes information about the student's identity, department, semester, course, marks obtained, grade, GPA/CGPA, scholarship status, and a few demographic attributes.

### 2.1 Dataset Fields

| Field                                        | Description                                         |
|----------------------------------------------|-----------------------------------------------------|
| Student_ID                                   | Unique identifier of the student                    |
| Name, Gender, Age                            | Basic demographic information of the student        |
| Department                                   | Academic department the student belongs to          |
| Admission_Year, Semester                     | Year of admission and current semester              |
| Course_Code, Course_Name                     | Course identifier and title                         |
| Credit_Hours                                 | Credit hours assigned to the course                 |
| Attendance_Percentage                        | Class attendance percentage for the course          |
| Midterm_Marks, Final_Marks, Assignment_Marks | Component-wise assessment marks                     |
| Total_Marks, Grade                           | Aggregated marks and the corresponding letter grade |

|                            |                                                                |
|----------------------------|----------------------------------------------------------------|
| GPA, CGPA                  | Course GPA and cumulative GPA                                  |
| Scholarship                | Whether the student receives a scholarship (Yes/No)            |
| Hostel_Resident            | Whether the student resides in a hostel                        |
| Family_Income_Range        | Household income bracket of the student's family               |
| Extracurricular_Activities | Whether the student participates in extracurricular activities |

## 2.2 Dataset Size

The dataset consists of 150,000 rows and 22 columns, stored as a single CSV file. This size is large enough that a single-machine, in-memory analysis becomes noticeably slower and less practical, which makes it a reasonable candidate for demonstrating distributed processing with Hadoop.

## 3. Dataset Preparation and Environment Setup

Rather than installing Hadoop directly on Windows, the cluster used for this project was set up using Docker. Docker allows a multi-node Hadoop environment (NameNode, DataNode, ResourceManager and NodeManager) to run in isolated containers on a single physical machine, which avoids many of the configuration issues associated with a native Hadoop installation on Windows, such as manually configuring environment variables, WinUtils binaries, and service paths. Each of these containers behaves like a separate machine on the network, even though all of them run on the same laptop, which makes it possible to observe the same kind of distributed behaviour (separate storage nodes, separate processing nodes, communication over a network) that a real multi-machine Hadoop cluster would exhibit.

This containerised setup also made the project considerably easier to manage on a day-to-day basis. Instead of a lengthy manual installation that has to be repeated whenever something goes wrong, the entire cluster could be brought down with a single command and rebuilt from scratch in the same state whenever needed. This proved particularly valuable during the early stages of the project, when the Hadoop and Pig configuration had to be adjusted and re-tested several times before the environment was stable enough to run the actual analyses. Any mistake made while experimenting with the cluster could be undone simply by removing the containers and starting new ones from the same image, without affecting the host machine's own operating system in any way.

The Hadoop images used were the bde2020 Hadoop images, orchestrated together using a docker-compose.yml file. Once the containers were running, the dataset file was copied from the host machine into the NameNode container using the docker cp command and then loaded into HDFS.

### 3.1 Cluster Architecture

The Hadoop images used were the bde2020 Hadoop images (Hadoop 3.2.1 on Java 8), orchestrated together using a single docker-compose.yml file. The compose file defines four separate services, each running in its own container but connected together on the same Docker network so that they can communicate using their container names as hostnames. The role of each service is summarised below.

| Container       | Role                                                                               | Exposed Port  |
|-----------------|------------------------------------------------------------------------------------|---------------|
| namenode        | Manages HDFS metadata; used as the entry point for running Hadoop and Pig commands | 9870 (Web UI) |
| datanode        | Stores the actual HDFS data blocks                                                 | 9864          |
| resourcemanager | Schedules YARN applications and allocates cluster resources                        | 8088 (Web UI) |
| nodemanager     | Executes MapReduce and Pig tasks on behalf of the ResourceManager                  | 8042          |

Persistent Docker volumes were attached to the namenode and datanode containers so that the contents of HDFS are preserved even if the containers are stopped and restarted, which was useful since the cluster needed to be brought down and back up several times over the course of the project without losing the uploaded dataset or the analysis outputs.

### 3.2 Steps Followed for Dataset Preparation

- Once the docker-compose.yml file was in place, the cluster was brought up and the dataset was prepared in HDFS through the following sequence of steps.
- Started Docker Desktop and confirmed that the Docker engine was running.
- Started the four-node Hadoop cluster from the project folder using docker-compose up -d, which pulled the required images on first run and then created and started all four containers.
- Verified that all four containers (namenode, datanode, resourcemanager, nodemanager) were running and reported a healthy status using docker ps.
- Opened an interactive shell inside the namenode container using docker exec -it namenode bash, since this is the container from which HDFS, MapReduce and Pig commands are issued.
- Copied university\_student\_dataset.csv from the local Windows machine into the namenode container's filesystem using the docker cp command, run from a separate PowerShell window on the host.
- Created an input directory inside HDFS and uploaded the dataset into it using hdfs dfs -mkdir -p /user/input followed by hdfs dfs -put.
- Verified that the upload was successful and that the file size matched the original CSV using hdfs dfs -ls /user/input.
- Installed Apache Pig 0.17.0 inside the namenode container (downloaded and extracted under /pig-0.17.0) and added it to the PATH, so that Pig scripts could be executed in the same container that hosts the HDFS input data.

### 3.3 Commands Used

```
docker-compose up -d
docker ps
docker cp university_student_dataset.csv namenode:/university_student_dataset.csv
docker exec -it namenode bash
hdfs dfs -mkdir -p /user/input
hdfs dfs -put /university_student_dataset.csv /user/input/
hdfs dfs -ls /user/input
```

This setup only needs to be performed once. For every subsequent working session, the cluster can simply be restarted with `docker-compose up -d`, and the dataset and all previous analysis outputs remain available in HDFS because of the persistent volumes described in **Section 3.1**.

```
Select Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Admin1> cd C:\Users\Admin1\bigdata-lab
PS C:\Users\Admin1\bigdata-lab> docker-compose up -d
WARN[0000] C:\Users\Admin1\bigdata-lab\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please
remove it to avoid potential confusion
[+] up 4/4
  ✓ Container namenode      Running      0.0s
  ✓ Container datanode      Running      0.0s
  ✓ Container resourceanager Running      0.0s
  ✓ Container nodemanager   Running      0.0s
PS C:\Users\Admin1\bigdata-lab> docker exec -it namenode bash
root@d202338f1859:/# hdfs dfs -ls /user/input
Found 2 items
-rw-r--r--   3 root supergroup          45 2026-09-12 22:38 /user/input/input.txt
-rw-r--r--  33 root supergroup 19698710 2026-09-14 22:04 /user/input/university_student_dataset.csv
root@d202338f1859:/# hdfs dfs -ls /user/
Found 10 items
drwxr-xr-x   0 root supergroup          0 2026-09-13 08:37 /user/data
drwxr-xr-x   0 root supergroup          0 2026-09-14 22:04 /user/input
drwxr-xr-x   0 root supergroup          0 2026-09-14 22:33 /user/output_mr1
drwxr-xr-x   0 root supergroup          0 2026-09-14 22:53 /user/output_mr2
drwxr-xr-x   0 root supergroup          0 2026-09-14 23:21 /user/output_mr3
drwxr-xr-x   0 root supergroup          0 2026-09-14 23:42 /user/output_pig1
drwxr-xr-x   0 root supergroup          0 2026-09-14 23:43 /user/output_pig2
drwxr-xr-x   0 root supergroup          0 2026-09-14 23:45 /user/output_pig3
drwxr-xr-x   0 root supergroup          0 2026-09-14 23:46 /user/output_pig4
drwxr-xr-x   0 root supergroup          0 2026-09-14 23:48 /user/output_pig5
root@d202338f1859:/#
```

*Docker containers started, dataset present in HDFS*

```
Windows PowerShell
root@d202338f1859:/# exit
exit

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug namenode
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Admin1\bigdata-lab> cd C:\Users\Admin1\bigdata-lab
PS C:\Users\Admin1\bigdata-lab> docker-compose stop
WARN[0000] C:\Users\Admin1\bigdata-lab\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please
remove it to avoid potential confusion
[+] stop 4/4
  ✓ Container nodemanager   Stopped      0.0s
  ✓ Container resourceanager Stopped      3.5s
  ✓ Container datanode      Stopped      3.4s
  ✓ Container namenode      Stopped      3.4s
PS C:\Users\Admin1\bigdata-lab> exit
```

*Stopping the Hadoop cluster containers using docker-compose stop after completing all analyses*

## 4. Hadoop and HDFS Setup

The Hadoop ecosystem used in this project runs entirely inside Docker containers and consists of the following core components.

### 4.1 Hadoop Components

- NameNode: Manages the HDFS namespace, metadata, and block locations.
- DataNode: Stores the actual data blocks that make up the files in HDFS.
- ResourceManager: Allocates cluster resources and schedules YARN applications.
- NodeManager: Executes tasks inside YARN containers on the worker node and monitors resource usage.

### 4.2 Hadoop Commands Used

```
docker-compose up -d
docker ps
docker exec -it namenode bash
hdfs dfs -mkdir -p /user/input
hdfs dfs -put university_student_dataset.csv /user/input/
hdfs dfs -ls /user/input
```

## 5. Summary of Data Analysis

Two Hadoop-based technologies were used to analyse the dataset: Java-style MapReduce, implemented through Hadoop Streaming with shell/awk mapper and reducer scripts, and Apache Pig, implemented using Pig Latin scripts executed in MapReduce mode. Both technologies ultimately run on top of the same underlying Hadoop engine and read the same input file from HDFS, but they represent two very different ways of expressing the same kind of computation.

| Technology                   | Number of Analyses |
|------------------------------|--------------------|
| MapReduce (Hadoop Streaming) | 3                  |
| Apache Pig                   | 5                  |
| Total                        | 8                  |

The MapReduce analyses demonstrate direct control over the map and reduce phases using mapper and reducer scripts. The Pig analyses demonstrate a higher-level, data-flow style of processing using operations such as FILTER, GROUP, FOREACH, ORDER and LIMIT, without needing to write a mapper or reducer manually.

## 6. MapReduce Analysis

MapReduce was implemented using the Hadoop Streaming API, which allows mapper and reducer logic to be written as ordinary shell scripts instead of Java classes. Each mapper reads the dataset line by line, splits it on the comma delimiter, and emits a key-value pair for the relevant field. Each reducer receives the intermediate key-value pairs after the shuffle and sort phase and aggregates them into the final result. Three MapReduce analyses were carried out on the dataset.

## 6.1 Department-wise Student Count

Objective: To determine how many students are enrolled in each department.

The mapper reads every record of the dataset (skipping the header row), extracts the Department field, and emits the department name with a value of 1. The reducer receives all the 1's for each department key and sums them to produce the total number of students in that department.

Mapper logic:

```
awk -F',' ' $1!="Student_ID" {print $5"\t1"}'
```

Reducer logic:

```
awk -F'\t' '{count[$1]+=$2}  
END {for (d in count) print d"\t"count[d}]'
```

Execution command:

```
hadoop jar hadoop-streaming-3.2.1.jar \  
-input /user/input/university_student_dataset.csv \  
-output /user/output_mr1 \  
-mapper mapper1.sh -reducer reducer_sum.sh \  
-file mapper1.sh -file reducer_sum.sh
```

Result: The output below shows the number of students enrolled in each of the fifteen departments.

| Department             | Number of Students |
|------------------------|--------------------|
| CSE                    | 9920               |
| Economics              | 9930               |
| BBA                    | 9929               |
| Mathematics            | 10076              |
| Architecture           | 10059              |
| Pharmacy               | 10120              |
| Law                    | 9878               |
| Civil Engineering      | 10034              |
| Public Health          | 10065              |
| Physics                | 9958               |
| Journalism             | 9992               |
| EEE                    | 10053              |
| Sociology              | 9848               |
| English                | 10154              |
| Mechanical Engineering | 9984               |

```
Windows PowerShell
Found 10 items
drwxr-xr-x - root supergroup 0 2026-09-13 08:37 /user/data
drwxr-xr-x - root supergroup 0 2026-09-14 22:04 /user/input
drwxr-xr-x - root supergroup 0 2026-09-14 22:33 /user/output_mr1
drwxr-xr-x - root supergroup 0 2026-09-14 22:53 /user/output_mr2
drwxr-xr-x - root supergroup 0 2026-09-14 23:21 /user/output_mr3
drwxr-xr-x - root supergroup 0 2026-09-14 23:42 /user/output_pig1
drwxr-xr-x - root supergroup 0 2026-09-14 23:43 /user/output_pig2
drwxr-xr-x - root supergroup 0 2026-09-14 23:45 /user/output_pig3
drwxr-xr-x - root supergroup 0 2026-09-14 23:46 /user/output_pig4
drwxr-xr-x - root supergroup 0 2026-09-14 23:48 /user/output_pig5
root@d202338f1859:/# hdfs dfs -cat /user/output_mr1/part-00000
2026-09-17 08:48:59,563 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHost
CSE 9920
Economics 9930
BBA 9929
Mathematics 10076
Architecture 10059
Pharmacy 10120
Law 9878
Civil Engineering 10034
Public Health 10065
Physics 9958
Journalism 9992
EEE 10053
Sociology 9848
English 10154
Mechanical Engineering 9984
root@d202338f1859:/#
```

Terminal output of MapReduce Analysis 1

## 6.2 Department-wise Average Total Marks

Objective: To calculate the average Total\_Marks obtained by students in each department, as an indicator of overall academic performance.

The mapper emits the department name together with the Total\_Marks value of each record. The reducer accumulates the sum of marks and the count of records for each department and computes the average by dividing the sum by the count.

Mapper logic:

```
awk -F',' ' $1!="Student_ID" {print $5"\t"$15}'
```

Reducer logic:

```
awk -F'\t' '{sum[$1]+=$2; count[$1]++}
END {for (d in sum)
    printf "%s\t%.2f\n", d, sum[d]/count[d]}'
```

Result: The average Total Marks per department are all fairly close to each other, ranging between roughly 67.4 and 67.7, which suggests that academic performance is fairly evenly distributed across departments in this dataset.

| Department             | Average Total Marks |
|------------------------|---------------------|
| CSE                    | 67.52               |
| Economics              | 67.43               |
| BBA                    | 67.42               |
| Mathematics            | 67.67               |
| Architecture           | 67.7                |
| Pharmacy               | 67.64               |
| Law                    | 67.55               |
| Civil Engineering      | 67.52               |
| Public Health          | 67.53               |
| Physics                | 67.54               |
| Journalism             | 67.65               |
| EEE                    | 67.46               |
| Sociology              | 67.52               |
| English                | 67.65               |
| Mechanical Engineering | 67.6                |

### 6.3 Scholarship Distribution

Objective: To determine how many students receive a scholarship and how many do not.

The mapper emits the value of the Scholarship field (Yes or No) with a count of 1 for every record. The reducer aggregates these counts to obtain the total number of students in each category.

Mapper logic:

```
awk -F',' ' $1!="Student_ID" {print $19"\t1"} '
```

The same generic sum-reducer used in Analysis 6.1 was reused here.

Result:

| Scholarship | Number of Students |
|-------------|--------------------|
| No          | 127425             |
| Yes         | 22575              |

Approximately 15% of students in the dataset receive a scholarship, while the remaining 85% do not.

```
Windows PowerShell
root@d202338f1859:/# hdfs dfs -cat /user/output_mr2/part-00000
2026-09-17 08:51:23,267 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false,
tTrusted = false
CSE      67.52
Economics 67.43
BBA      67.42
Mathematics 67.67
Architecture 67.70
Pharmacy 67.64
Law      67.55
Civil Engineering 67.52
Public Health 67.53
Physics 67.54
Journalism 67.65
EEE      67.46
Sociology 67.52
English 67.65
Mechanical Engineering 67.60
root@d202338f1859:/# hdfs dfs -cat /user/output_mr3/part-00000
2026-09-17 08:51:25,540 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false,
tTrusted = false
No      127425
Yes     22575
root@d202338f1859:/#
```

*Terminal output of MapReduce Analysis 2 and Analysis 3*

## 7. Pig Analysis

Apache Pig was used to perform five higher-level analyses on the same dataset. Pig scripts were written in Pig Latin and executed in MapReduce mode using the command `pig -x mapreduce script.pig`. Each script loads the CSV file with an explicit schema, filters out the header row, and then applies grouping, aggregation, ordering or limiting operations as required by the analysis.

Common load statement used at the start of every script:

```
data = LOAD '/user/input/university_student_dataset.csv'
      USING PigStorage(',') AS
      (student_id:chararray, name:chararray, gender:chararray,
       age:int, department:chararray, admission_year:int,
       semester:int, course_code:chararray, course_name:chararray,
       credit_hours:float, attendance_percentage:float,
       midterm_marks:float, final_marks:float, assignment_marks:float,
       total_marks:float, grade:chararray, gpa:float, cgpa:float,
       scholarship:chararray, hostel_resident:chararray,
       family_income_range:chararray, extracurricular_activities:chararray);

filtered = FILTER data BY student_id != 'Student_ID';
```

## 7.1 Average CGPA by Department

Objective: To identify which departments, have the highest average CGPA among their students.

```
grouped = GROUP filtered BY department;  
avg_cgpa = FOREACH grouped GENERATE  
            group AS department, AVG(filtered.cgpa) AS avg_cgpa;  
result = ORDER avg_cgpa BY avg_cgpa DESC;  
STORE result INTO '/user/output_pig1' USING PigStorage(',');
```

Result: Architecture, Mechanical Engineering and Physics have the highest average CGPA, while Economics has the lowest, although the overall spread across departments is narrow (roughly 3.19 to 3.21).

| Department             | Average CGPA |
|------------------------|--------------|
| Architecture           | 3.2104       |
| Mechanical Engineering | 3.2074       |
| Physics                | 3.2073       |
| Law                    | 3.2072       |
| Journalism             | 3.2064       |
| Mathematics            | 3.2053       |
| Pharmacy               | 3.2043       |
| EEE                    | 3.2037       |
| BBA                    | 3.2022       |
| English                | 3.2011       |
| Public Health          | 3.1997       |
| CSE                    | 3.1996       |
| Civil Engineering      | 3.1977       |
| Sociology              | 3.1953       |
| Economics              | 3.1900       |

## 7.2 Grade Distribution

Objective: To find out how many students fall into each letter grade category across the entire dataset.

```
grouped = GROUP filtered BY grade;  
grade_count = FOREACH grouped GENERATE  
            group AS grade, COUNT(filtered) AS total_students;  
result = ORDER grade_count BY total_students DESC;  
STORE result INTO '/user/output_pig2' USING PigStorage(',');
```

Result: The grade B+ is the most common grade in the dataset, followed by A- and B, while very few students fall into the D and F categories.

| Grade | Number of Students |
|-------|--------------------|
| B+    | 26185              |
| A-    | 24352              |
| B     | 23620              |
| A+    | 20137              |
| A     | 18795              |
| B-    | 17202              |
| C+    | 10844              |
| C     | 5445               |
| D     | 2322               |
| F     | 1098               |

```

Windows PowerShell
root@d202338f1859:/# hdfs dfs -cat /user/output_pig1/part-r-00000
2026-09-17 08:54:08,927 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
Architecture,3.2103777725332114
Mechanical Engineering,3.207439904938861
Physics,3.2072976504806614
Law,3.207198826327813
Journalism,3.2064481604793253
Mathematics,3.2053027005321573
Pharmacy,3.204315218360167
EEE,3.2037163046253037
BBA,3.2021613454659867
English,3.201132559578705
Public Health,3.199673124599667
CSE,3.1996078630145113
Civil Engineering,3.1976818822661013
Sociology,3.1952914303555633
Economics,3.1900140992661212
root@d202338f1859:/# hdfs dfs -cat /user/output_pig2/part-r-00000
2026-09-17 08:54:10,970 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
B+,26185
A-,24352
B,23620
A+,20137
A,18795
B-,17202
C+,10844
C,5445
D,2322
F,1098

```

Terminal output of Pig Analysis 1 & Analysis 2

### 7.3 Top 5 Departments by Average Attendance

Objective: To find the five departments with the highest average class attendance.

```

grouped = GROUP filtered BY department;
avg_att = FOREACH grouped GENERATE
    group AS department,
    AVG(filtered.attendance_percentage) AS avg_attendance;
sorted = ORDER avg_att BY avg_attendance DESC;
top5 = LIMIT sorted 5;
STORE top5 INTO '/user/output_pig3' USING PigStorage(',');

```

Result: EEE recorded the highest average attendance, closely followed by Architecture, Public Health, BBA and Pharmacy. The differences between the top five departments are small, all lying above 81.7%.

| Department    | Average Attendance (%) |
|---------------|------------------------|
| EEE           | 81.7977                |
| Architecture  | 81.7830                |
| Public Health | 81.7801                |
| BBA           | 81.7723                |
| Pharmacy      | 81.7513                |

## 7.4 Scholarship Count by Gender

Objective: To compare scholarship distribution between male and female students.

```
grouped = GROUP filtered BY (gender, scholarship);
result = FOREACH grouped GENERATE
    group.gender AS gender, group.scholarship AS scholarship,
    COUNT(filtered) AS total;
STORE result INTO '/user/output_pig4' USING PigStorage(',');
```

Result: Both male and female students show a similar scholarship rate of roughly 15%, indicating that scholarship allocation in this dataset does not appear to favour one gender over the other.

| Gender | Scholarship | Number of Students |
|--------|-------------|--------------------|
| Male   | No          | 69958              |
| Male   | Yes         | 12442              |
| Female | No          | 57467              |
| Female | Yes         | 10133              |

## 7.5 Average Total Marks by Semester

Objective: To observe whether academic performance, measured by Total\_Marks, changes as students progress through semesters.

```
grouped = GROUP filtered BY semester;
avg_marks = FOREACH grouped GENERATE
    group AS semester, AVG(filtered.total_marks) AS avg_total_marks;
result = ORDER avg_marks BY semester ASC;
STORE result INTO '/user/output_pig5' USING PigStorage(',');
```

Result: Average marks remain fairly stable across all eight semesters, staying within a narrow band of about 67.4 to 67.7. This suggests that, at least in this dataset, overall academic performance does not vary significantly as students move from earlier to later semesters.

| Semester | Average Total Marks |
|----------|---------------------|
| 1        | 67.58               |
| 2        | 67.53               |
| 3        | 67.65               |
| 4        | 67.58               |
| 5        | 67.44               |
| 6        | 67.54               |
| 7        | 67.53               |
| 8        | 67.61               |

```
Windows PowerShell
root@d202338f1859:/# hdfs dfs -cat /user/output_pig3/part-r-00000
2026-09-17 08:59:25,961 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
EEE,81.79773200648016
Architecture,81.78303009611153
Public Health,81.78005964806941
BBA,81.772273144053
Pharmacy,81.7512944225266
root@d202338f1859:/# hdfs dfs -cat /user/output_pig4/part-r-00000
2026-09-17 08:59:28,120 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
Male,No,69958
Male,Yes,12442
Female,No,57467
Female,Yes,10133
root@d202338f1859:/# hdfs dfs -cat /user/output_pig5/part-r-00000
2026-09-17 08:59:30,322 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
1,67.58199415077588
2,67.5332426532158
3,67.65208064392688
4,67.58403814849142
5,67.44206191011816
6,67.54164280104023
7,67.52831892910095
8,67.61421131834287
root@d202338f1859:/#
```

Terminal output of Pig Analysis 3, Analysis 4 & Analysis 5

7.6 HDFS Output Directory Structure

All MapReduce and Pig outputs were written to separate directories under /user/ in HDFS, which keeps the results of each analysis isolated and easy to inspect individually using the HDFS web interface or the command line.

## Browse Directory





Show  entries

Search:

| <input type="checkbox"/> | Permission                 | Owner                | Group                      | Size | Last Modified | Replication       | Block Size | Name                        |                                                                                     |
|--------------------------|----------------------------|----------------------|----------------------------|------|---------------|-------------------|------------|-----------------------------|-------------------------------------------------------------------------------------|
| <input type="checkbox"/> | <a href="#">drwxr-xr-x</a> | <a href="#">root</a> | <a href="#">supergroup</a> | 0 B  | Sep 13 14:37  | <a href="#">0</a> | 0 B        | <a href="#">data</a>        |  |
| <input type="checkbox"/> | <a href="#">drwxr-xr-x</a> | <a href="#">root</a> | <a href="#">supergroup</a> | 0 B  | Sep 15 04:04  | <a href="#">0</a> | 0 B        | <a href="#">input</a>       |  |
| <input type="checkbox"/> | <a href="#">drwxr-xr-x</a> | <a href="#">root</a> | <a href="#">supergroup</a> | 0 B  | Sep 15 04:33  | <a href="#">0</a> | 0 B        | <a href="#">output_mr1</a>  |  |
| <input type="checkbox"/> | <a href="#">drwxr-xr-x</a> | <a href="#">root</a> | <a href="#">supergroup</a> | 0 B  | Sep 15 04:53  | <a href="#">0</a> | 0 B        | <a href="#">output_mr2</a>  |  |
| <input type="checkbox"/> | <a href="#">drwxr-xr-x</a> | <a href="#">root</a> | <a href="#">supergroup</a> | 0 B  | Sep 15 05:21  | <a href="#">0</a> | 0 B        | <a href="#">output_mr3</a>  |  |
| <input type="checkbox"/> | <a href="#">drwxr-xr-x</a> | <a href="#">root</a> | <a href="#">supergroup</a> | 0 B  | Sep 15 05:42  | <a href="#">0</a> | 0 B        | <a href="#">output_pig1</a> |  |
| <input type="checkbox"/> | <a href="#">drwxr-xr-x</a> | <a href="#">root</a> | <a href="#">supergroup</a> | 0 B  | Sep 15 05:43  | <a href="#">0</a> | 0 B        | <a href="#">output_pig2</a> |  |
| <input type="checkbox"/> | <a href="#">drwxr-xr-x</a> | <a href="#">root</a> | <a href="#">supergroup</a> | 0 B  | Sep 15 05:45  | <a href="#">0</a> | 0 B        | <a href="#">output_pig3</a> |  |
| <input type="checkbox"/> | <a href="#">drwxr-xr-x</a> | <a href="#">root</a> | <a href="#">supergroup</a> | 0 B  | Sep 15 05:46  | <a href="#">0</a> | 0 B        | <a href="#">output_pig4</a> |  |
| <input type="checkbox"/> | <a href="#">drwxr-xr-x</a> | <a href="#">root</a> | <a href="#">supergroup</a> | 0 B  | Sep 15 05:48  | <a href="#">0</a> | 0 B        | <a href="#">output_pig5</a> |  |

Showing 1 to 10 of 10 entries

Previous
1
Next

*HDFS Browse Directory page (localhost:9870) showing all output folders under /user*

|                   |                                   |
|-------------------|-----------------------------------|
| /user/input       | (source dataset)                  |
| /user/output_mr1  | (Department-wise student count)   |
| /user/output_mr2  | (Department-wise average marks)   |
| /user/output_mr3  | (Scholarship distribution)        |
| /user/output_pig1 | (Average CGPA by department)      |
| /user/output_pig2 | (Grade distribution)              |
| /user/output_pig3 | (Top 5 departments by attendance) |
| /user/output_pig4 | (Scholarship by gender)           |
| /user/output_pig5 | (Average marks by semester)       |

## 8. Discussion

This section reflects what was observed while carrying out the project, and the practical differences between the two processing approaches used.

### 8.1 HDFS Storage

Storing the dataset in HDFS rather than on a local disk made it straightforward to reuse the same input file across all eight analyses without moving or duplicating data. Even though 150,000 rows is not an enormous dataset by Big Data standards, it was large enough to clearly demonstrate the map-reduce-shuffle-sort cycle and to make the effect of grouping and ordering operations visible in the execution time of the jobs.

## 8.2 MapReduce versus Pig

Writing the MapReduce jobs using Hadoop Streaming with awk scripts required thinking explicitly about how each line of the CSV file should be split into a key and a value, and how the reducer should combine the values for each key. This gives fine control over the processing logic but means that even a simple aggregation, such as counting students per department, needs a mapper and a matching reducer to be written and tested separately.

In contrast, the same kind of aggregation in Pig could be expressed in only a few lines using GROUP and FOREACH, without writing any mapper or reducer at all. Pig scripts were also easier to modify - for example, changing Analysis 7.1 from grouping by department to grouping by semester only required changing the GROUP BY clause and one field name, whereas the same change in the MapReduce version would require editing the awk expression in the mapper script.

On the other hand, the MapReduce approach made it clearer exactly what work was happening in the mapper and what was happening in the reducer, which was useful for understanding how Hadoop actually distributes and combines data internally.

## 8.3 Observations from the Results

Across all three MapReduce analyses and five Pig analyses, department-level averages (Total Marks, CGPA, Attendance) were found to be very close to one another, typically differing by less than one percentage point or half a grade point. This suggests that the dataset was generated with a fairly uniform distribution of academic performance across departments rather than favouring any particular department. The grade distribution (Analysis 7.2), however, does show a clear pattern, with the middle grades (B+, A-, B) being far more common than the extreme grades (D, F), which resembles a typical bell-shaped distribution of student performance.

The scholarship analyses (6.3 and 7.4) show that scholarships are relatively rare overall (about 15% of students) and that this rate is essentially the same for male and female students, indicating no visible gender bias in scholarship allocation within this dataset.

## 8.4 Practical Challenges

The main practical challenges in this project were not related to the analysis logic itself, but to the environment setup. Running a multi-container Hadoop cluster on Windows through Docker required resolving several issues, including outdated package repositories inside the containers (which affected installing tools such as wget), and Hadoop Streaming requiring the exact path to the streaming JAR file to be located manually. Once these environment issues were resolved, all eight analyses ran successfully and produced consistent results.

## 9. Conclusion

This project demonstrated how a large tabular dataset of university student records can be processed and analysed using Hadoop-based Big Data tools running on a Dockerised cluster. Three MapReduce jobs, implemented through Hadoop Streaming, were used to count students by department, compute average marks by department, and determine the scholarship distribution. Five Apache Pig scripts were then used to extend the analysis further, covering average CGPA by department, grade distribution, top departments by attendance, scholarship distribution by gender, and average marks by semester.

The results show an even distribution of academic performance across departments and semesters, with the main variation appearing in the grade distribution and in a scholarship rate that stays balanced across genders. The project also highlighted the trade-off between MapReduce, which offers explicit control over mapping and reducing logic, and Pig, which expresses the same analysis far more concisely through a higher-level, SQL-like syntax.

Overall, the project provided hands-on experience with HDFS, Hadoop Streaming MapReduce, and Apache Pig, illustrating how these tools can be used together to explore a real-world-style dataset efficiently.